Papilot: Reviewable Agent Edits for LaTeX Manuscripts

The Papilot Authors

Abstract

Papilot is a personal workspace for revising LaTeX papers with an AI agent and reviewing every resulting change as a PDF difference before it reaches the manuscript. Each editing task runs in an isolated Git worktree, may be driven by Codex, Claude, Antigravity or Kimi Code, and proceeds in review rounds in which the author accepts, rejects or comments on individual changes. This paper describes the system through a sequence of small formal models. We define edits and review rounds as operators on a document, prove that compatible edits commute and can be rebased, and bound the number of rounds a task needs. We estimate how often concurrent tasks conflict, both for uniformly placed and for clustered edits. We then treat the geometry of the PDF view: the chain of matrices from a glyph to a pixel, why a line-wide text layer drifts from the printed words, and how per-word runs bound that drift. We show how a mark on the page becomes a source location, model agent scheduling as a queue and usage limits as a compound Poisson process, and give the index algebra behind partial commits. A model-based evaluation tabulates the resulting quantities. The paper doubles as Papilot's example manuscript: every equation in it is a place to try a review.

Contents

1 Introduction **4**
- 1.1 Three commitments . . . 4
- 1.2 What this paper does . . . 4
- 1.3 A note on this document . . . 5

2 Using Papilot **5**
- 2.1 Running it . . . 5
- 2.2 A task from start to finish . . . 5
- 2.3 Reading the paper . . . 6
- 2.4 Notes on the PDF . . . 6
- 2.5 The Git panel . . . 6
- 2.6 Agents . . . 6
- 2.7 Working from another device . . . 6
- 2.8 Configuration . . . 7

3 Background **7**
- 3.1 The LaTeX build as a fixed point . . . 7
- 3.2 Text in a PDF . . . 7
- 3.3 Git worktrees . . . 8
- 3.4 SyncTeX . . . 8

4 A model of review rounds **8**
4.1 Documents and edits . . . 8
4.2 Rebasing edits . . . 9
4.3 Decisions and rounds . . . 10
4.4 How many rounds? . . . 10

5 Concurrent tasks and conflicts **11**
5.1 Uniformly placed edits . . . 11
5.2 How much overlaps . . . 11
5.3 Clustered edits . . . 11

6 Building the PDF **12**
6.1 Snapshots of the working tree . . . 12
6.2 When to build . . . 12
6.3 Reviewing changed pages . . . 13

7 Geometry of the PDF view **13**
7.1 From a glyph to a pixel . . . 13
7.2 How TEX spaces a line . . . 13
7.3 Why a line-wide text layer drifts . . . 14
7.4 Per-word runs . . . 15

8 Notes on the PDF **15**
8.1 From a mark to source lines . . . 16
8.2 Settling a selection . . . 16
8.3 When is a match unique? . . . 16
8.4 Selecting whole words . . . 17

9 Agents and scheduling **17**
9.1 One protocol for every agent . . . 17
9.2 Effort and cost . . . 17
9.3 Queueing . . . 18
9.4 Approvals . . . 18

10 Usage windows **18**
10.1 Utilisation . . . 18
10.2 Will the limit be reached? . . . 19

11 Closing the loop with Git **19**
11.1 Committing chosen files . . . 19
11.2 Synchronising with the remote . . . 20

12 Security **20**
12.1 Requests . . . 20
12.2 Agents . . . 20

13 A model-based evaluation **21**
13.1 Rounds per task . . . 21
13.2 Conflicts . . . 21
13.3 Waiting for a slot . . . 21
13.4 Selection error . . . 22

14 Related work **22**

15 Discussion and conclusion **23**
15.1 Limitations . . . 23
15.2 Conclusion . . . 23

A Proofs **23**
A.1 Expected rounds . . . 23
A.2 Mid-line drift . . . 23
A.3 Partial commits . . . 24

B Notation and keyboard **24**
B.1 Notation . . . 24
B.2 Keyboard . . . 25

1 Introduction

Writing a research paper is a long sequence of small decisions. A sentence is shortened, a definition is moved before its first use, a symbol is renamed so that it no longer collides with another, a proof gains the step a referee asked for. An agent that edits LaTeX can propose many of these decisions quickly, but a proposal is only useful when the author can see what it changes in the typeset paper, decide, and say why. Papilot is built around that loop.

1.1 Three commitments

Papilot organises the loop around three commitments.

(C1) *Isolation.* Every task edits its own Git worktree, never the manuscript checkout, so that unreviewed text cannot leak into the paper.

(C2) *Typeset review.* Changes are reviewed as differences between two compiled PDFs, with the LaTeX source one click away.

(C3) *Explicit acceptance.* Only accepted changes are merged into the working tree, and every decision can be undone.

The first commitment is about safety, the second about legibility and the third about authority. Together they turn an agent from a co-author that writes into the manuscript into an assistant that proposes to it.

1.2 What this paper does

The rest of this paper makes the commitments precise and studies their consequences with small, explicit models. Our contributions are:

- an algebra of edits and review rounds, with a proof that compatible edits commute and can be rebased onto each other (section 4), and tight bounds on the number of rounds a task needs;
- an estimate of the conflict rate between concurrent tasks under uniform and clustered edits (section 5);
- a content-addressed build scheme in which undoing a decision never recompiles (section 6);
- a geometric account of the PDF view, explaining why a line-wide text layer drifts from the printed words and proving that per-word runs bound the drift (section 7);
- an algorithm that turns a mark on the page into source lines, and an analysis of when its matches are unique (section 8);
- queueing and usage models for agents (sections 9 and 10), the index algebra behind partial commits (section 11) and a security model (section 12);
- a model-based evaluation that tabulates the quantities above (section 13).

Section 2 describes Papilot as an author uses it, section 3 collects the background on TeX, PDF, Git and SyncTeX that the later sections rely on, section 14 discusses related work, and section 15 concludes. Longer proofs are in section A; section B lists the notation and the keyboard shortcuts.

1.3 A note on this document

This paper is also Papilot's example manuscript. It is deliberately dense with displayed mathematics, theorems and cross-references, because those are the places where an agent's edit is easiest to get subtly wrong and a typeset review most useful. The reader is invited to start a task against any equation and to review the result.

2 Using Papilot

This section describes Papilot as an author meets it. The later sections explain why it behaves as it does.

2.1 Running it

Papilot needs Node.js 22.22 or later, Git, a TEX installation with `latexmk` and `synctex`, and a signed-in agent on the host. It is installed and started with

```
npm install -g papilot
papilot
```

and then opened at `http://127.0.0.1:7274`. *Add paper* registers a LATEX Git repository on the host; later papers are added and switched from the paper menu at the top of the sidebar. A repository must have at least one commit and compile with `latexmk`; a custom root document is named in a `.latex-review.json` file. To try a sample first, `papilot demo` copies this paper into a separate repository and registers it as *Papilot paper*, without touching any existing manuscript.

2.2 A task from start to finish

1. Select *New task*, describe the revision and press Enter. The first line of the prompt becomes the task's title, and the task runs on the paper selected in the sidebar. Below the prompt, choose the agent, its model and its reasoning effort; Papilot remembers all three for the next task.

2. Follow the conversation and answer the agent's questions or permission requests. When both execution slots are busy, further tasks queue (section 9).

3. When the agent finishes, Papilot builds the PDF review automatically. Accept, reject or comment on each change in the typeset review.

4. Send the saved comments, or write further instructions in the conversation. Either starts the next round, which carries the review feedback with it (section 4).

While the agent works, the composer shows a stop button that pauses the task. The desktop workspace offers three views, *Review*, *Split* and *Chat*, and remembers the choice; on a phone the header switches between the review and the chat. The review toolbar shows the round's progress, accepts or rejects every undecided change at once, opens the full PDF of the round, builds the review again, and opens it in a separate tab. *Accept all* and *Reject all* act only on changes not yet decided, and one undo reverses the whole batch.

Conversation drafts are saved per task in the browser; Enter sends, Shift+Enter adds a line, and an input method's composition is never sent by Enter. Closing the browser does not stop a

run. After a restart of the host process, turns that were in flight are marked interrupted and can be continued, and queued messages are kept. If the outcome of a request is uncertain, the task offers *Check status*, and further decisions wait until it is resolved.

2.3 Reading the paper

PDF, under the paper in the sidebar, shows the manuscript as it is on disk, including accepted but uncommitted changes (section 6). It behaves like a local PDF reader: continuous pages with crisp text at any zoom, selectable text, working links and cross-references, and search across the whole document.

- *Contents* lists the PDF's bookmarks, which `hyperref` writes for every section, highlights the section being read, and jumps to any entry. The same list opens from the toolbar.
- A page number or the arrows change page; the zoom buttons, the zoom menu, the keyboard and a pinch change the zoom, which is remembered.
- Find (Cmd+F) searches every page, with Enter and Shift+Enter stepping through the matches.

2.4 Notes on the PDF

To ask for a change directly on the PDF, select text and choose *Add note*, or mark a region around a figure or an equation, then describe the change. The selection follows the printed words exactly (section 7), each note is settled on the source lines that produced it (section 8), and every pending note is sent to the agent as one task.

2.5 The Git panel

Beside the PDF, a small source-control panel shows the branch and how it compares with its upstream, the last commit, and the uncommitted changes with their line counts. Selecting a file shows its difference. Ticking files, writing a message and pressing Enter commits exactly those files as they are on disk; fetch, a fast-forward pull and push keep the branch in step with its remote (section 11).

2.6 Agents

Each task runs on one agent: Codex through its app server, Claude through the Claude Agent SDK, Antigravity through its command line, or Kimi Code through ACP. For Kimi, install the CLI and run `kimi login`. Every agent uses its own login on the host, and an agent that is not installed or signed in is listed as unavailable. When an agent reports usage, the new-task page shows each usage window with how much is used and when it resets (section 10).

2.7 Working from another device

Papilot binds only to `127.0.0.1`. To reach it from a phone or another computer, it is published on a private Tailscale network, and the server checks the host, the browser's origin and the Tailscale identity of every remote request (section 12):

```
export PAPILOT_PUBLIC_ORIGIN=https://your-mac.your-tailnet.ts.net:8443
export PAPILOT_TAILSCALE_LOGIN=your-tailscale-login
papilot service install
tailscale serve --bg --https=8443 http://127.0.0.1:7274
```

A background service starts Papilot after login, restarts it if it exits and keeps the computer from sleeping while it runs. Without Tailscale, an SSH tunnel serves the same purpose.

2.8 Configuration

Variable	Purpose
`PAPILOT_PORT`	local HTTP port, 7274 by default
`PAPILOT_DATA`	state and generated example, `~/.papilot` by default
`PAPILOT_CODEX_BIN`	Codex executable
`ANTHROPIC_API_KEY`	API key for Claude tasks
`PAPILOT_KIMI_BIN`	Kimi Code executable
`PAPILOT_AGY_BIN`	Antigravity command line
`PAPILOT_AGY_PROXY`	HTTP proxy used only for Antigravity
`PAPILOT_REVIEW_ENGINE`	review engine directory
`PAPILOT_PUBLIC_ORIGIN`	exact browser origin for remote access
`PAPILOT_TAILSCALE_LOGIN`	Tailscale login allowed to connect

3 Background

This section fixes notation for the four systems Papilot sits between: the TEX build, the PDF format, Git, and SyncTeX.

3.1 The LATEX build as a fixed point

LATEX resolves cross-references, citations and the table of contents by reading auxiliary data written by the previous run [12]. Let x_k be the auxiliary state after run k and F the map computed by one run, so that

$$x_{k+1} = F(x_k), \qquad x_0 = \varnothing. \tag{3.1}$$

A build tool such as `latexmk` repeats runs until a fixed point $x^\star = F(x^\star)$ is reached or a limit of K runs is hit (five by default). A single run suffices when $F(\varnothing) = \varnothing$, which is the case only for documents without references. A typical paper converges in two or three runs: the first records labels, the second resolves them, and a third is needed only when resolving them moved a page break.

Remark 3.1. F need not be a contraction. A reference whose resolution lengthens a line can push a label onto the next page, whose page number shortens the line again; the sequence x_k then alternates between two states and the run limit ends the build. Such documents are rare, but they are why a build key must include the build settings and not only the sources (section 6).

3.2 Text in a PDF

A PDF page places glyphs with a chain of matrices [8]. The text state contains the font size T_{fs}, the horizontal scaling T_h, the rise T_{rise}, the character and word spacings T_c and T_w, and the text matrix T_m. A glyph is drawn in user space through the text rendering matrix

$$T_{rm} = \begin{pmatrix} T_{fs}T_h & 0 & 0 \\ 0 & T_{fs} & 0 \\ 0 & T_{\text{rise}} & 1 \end{pmatrix} \times T_m \times \text{CTM}, \tag{3.2}$$

where CTM is the current transformation matrix. After each glyph with horizontal advance w_0 (in thousandths of a text space unit) and a positioning adjustment T_j, the text matrix moves by

$$t_x = \Big((w_0 - \tfrac{T_j}{1000}) T_{fs} + T_c + T_w \Big) T_h, \qquad T_m \leftarrow \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & 0 & 1 \end{pmatrix} T_m, \tag{3.3}$$

with T_w applied only to the single-byte space character. TEX rarely uses T_w; it positions words with explicit adjustments T_j, which is why a PDF produced from TEX often contains no space characters at all.

3.3 Git worktrees

A Git repository is a store of immutable objects (blobs, trees and commits) addressed by their hashes, together with a set of references [3]. A *worktree* is a checkout with its own working files W_i, index I_i and head HEAD_i, sharing the object store with every other worktree of the same repository:

$$\mathcal{R} = (\mathcal{O},\ \mathrm{refs}), \qquad w_i = (W_i,\ I_i,\ \mathrm{HEAD}_i), \quad i = 0, 1, \ldots, t. \tag{3.4}$$

The manuscript checkout is w_0; each task $i \geq 1$ gets its own w_i on its own branch. Because objects are shared, creating a worktree costs only the working files, and because indexes are separate, a task can stage, diff and build without touching w_0.

3.4 SyncTeX

SyncTeX records, while TEX typesets, where each box, glue and kern came from: an input file, a line and a column [13]. Positions are recorded in TEX's scaled points. With

$$1\,\mathrm{pt} = \tfrac{1}{72.27}\,\mathrm{in}, \qquad 1\,\mathrm{bp} = \tfrac{1}{72}\,\mathrm{in}, \qquad 1\,\mathrm{sp} = 2^{-16}\,\mathrm{pt}, \tag{3.5}$$

a horizontal position h in scaled points becomes PDF points as

$$x = (h + h_{\mathrm{origin}}) \cdot 2^{-16} \cdot \frac{72}{72.27}, \tag{3.6}$$

where h_{origin} is the one-inch offset of TEX's origin from the page edge, expressed in scaled points. A reverse query ("edit") returns the record nearest to a point on the page; a forward query ("view") returns the boxes produced by a source line. Papilot uses only reverse queries.

4 A model of review rounds

4.1 Documents and edits

We view a manuscript as a finite sequence of source characters over an alphabet Σ, $D \in \Sigma^*$, of length $n = |D|$. An *edit* replaces a half-open range of the source by new text.

Definition 4.1 (Edit)**.** An edit is a triple $e = (a, b, w)$ with $0 \leq a \leq b \leq n$ and $w \in \Sigma^*$. Applying e to D gives

$$D \oplus e = D_{[0,a)}\, w\, D_{[b,n)}, \tag{4.1}$$

whose length is $n + \delta(e)$ with *shift* $\delta(e) = |w| - (b - a)$.

A task proposes a *revision* $\Delta = \{e_1, \dots, e_m\}$, a finite set of edits against the same base document. Papilot groups nearby edits into *changes*, the units the author decides on.

Definition 4.2 (Compatibility)**.** Two edits $e = (a, b, w)$ and $e' = (a', b', w')$ are *compatible*, written $e \perp e'$, when their ranges are disjoint and not adjacent:

$$e \perp e' \iff b < a' \text{ or } b' < a. \tag{4.2}$$

A revision is compatible when its edits are pairwise compatible.

For a compatible revision we apply edits from right to left, so that earlier offsets remain valid:

$$D \oplus \Delta = \big(\cdots \big((D \oplus e_{\sigma(1)}) \oplus e_{\sigma(2)}\big) \cdots\big) \oplus e_{\sigma(m)}, \qquad a_{\sigma(1)} > a_{\sigma(2)} > \cdots > a_{\sigma(m)}. \tag{4.3}$$

Proposition 4.3 (Compatible edits commute)**.** *If $e \perp e'$, then applying them right to left in either listing gives the same document, and the result of eq. (4.3) does not depend on how the remaining edits of a compatible revision are ordered.*

Proof. Assume without loss of generality $b < a'$. Applying e' first changes only positions at or after a', so the range $[a, b)$ and every offset before it are unchanged, and applying e afterwards gives $D_{[0,a)}\, w\, D_{[b,a')}\, w'\, D_{[b',n)}$. Applying e first shifts every later offset by $\delta(e)$; rewriting e' as $(a' + \delta(e), b' + \delta(e), w')$ yields the same string. Induction on the number of edits, ordering them by start offset, gives the general statement. □

4.2 Rebasing edits

When one task's edit e has been accepted, another task's edit e', written against the old document, must be moved onto the new one. The *transformation* of e' past e is

$$\tau(e', e) = \begin{cases} (a', b', w'), & b' < a, \\ (a' + \delta(e),\ b' + \delta(e),\ w'), & b < a', \\ \bot, & \text{otherwise}, \end{cases} \tag{4.4}$$

where $\bot$ signals a conflict that the author must resolve.

Lemma 4.4 (Convergence of rebased edits)**.** *If $e \perp e'$, then*

$$D \oplus e \oplus \tau(e', e) = D \oplus e' \oplus \tau(e, e'). \tag{4.5}$$

Proof. If $b < a'$, then $\tau(e, e') = e$ and $\tau(e', e)$ is e' shifted by $\delta(e)$, so the left side is the second computation in the proof of proposition 4.3 and the right side the first. The case $b' < a$ is symmetric. □

Equation (4.5) is the convergence property known from operational transformation [4]. Papilot needs only the compatible case: an incompatible pair is never merged automatically.

4.3 Decisions and rounds

Let U be the set of changes in a round. A *decision state* is a map

$$\theta : U \to \{\mathsf{pending}, \mathsf{accepted}, \mathsf{rejected}, \mathsf{commented}\}, \tag{4.6}$$

and the accepted part of a revision is $\Delta^\theta = \{e \in \Delta : \theta(u(e)) = \mathsf{accepted}\}$. The manuscript after round t is

$$D_{t+1} = \begin{cases} D_t \oplus \Delta_t^{\theta_t}, & \text{if } \Delta_t^{\theta_t} \text{ applies cleanly to } D_t, \\ D_t, & \text{otherwise, and the task is asked for a new proposal.} \end{cases} \tag{4.7}$$

Rejected and commented changes return to the agent as feedback bound to their round, file and lines; the agent must not reintroduce a rejected edit unless it is explicitly asked to.

Actions and undo. An *action* $\alpha = (\iota, u, d)$ carries an identifier ι, a change u and a decision d. Applying it to a state gives $\theta \cdot \alpha = \theta[u \mapsto d]$, and the history is a stack $\alpha_1, \dots, \alpha_k$. Two properties make decisions safe over an unreliable network:

$$(\theta \cdot \alpha) \cdot \alpha = \theta \cdot \alpha \qquad \text{(idempotence: a retried write changes nothing),} \tag{4.8}$$

$$\mathrm{undo}(\theta_0 \cdot \alpha_1 \cdots \alpha_k) = \theta_0 \cdot \alpha_1 \cdots \alpha_{k-1} \qquad \text{(undo reverses exactly the last action).} \tag{4.9}$$

An undo request names the action it reverses, so a retried undo cannot reverse a different one.

4.4 How many rounds?

Suppose that each change is accepted in a round independently with probability $p \in (0, 1]$, and otherwise revised and proposed again. The number of rounds R until a single change is accepted is geometric,

$$\mathbb{P}(R = r) = (1-p)^{r-1}p, \qquad \mathbb{E}[R] = \frac{1}{p}, \qquad \mathrm{Var}(R) = \frac{1-p}{p^2}. \tag{4.10}$$

For m independent changes the task ends when the last one is accepted, after $R_{\max} = \max_i R_i$ rounds.

Proposition 4.5 (Expected rounds)**.**

$$\mathbb{E}[R_{\max}] = \sum_{k=1}^{m} (-1)^{k+1} \binom{m}{k} \frac{1}{1-(1-p)^k}. \tag{4.11}$$

The proof is in section A. For small p the sum behaves like $\frac{1}{p} H_m$, where $H_m = \sum_{k=1}^{m} 1/k \approx \ln m + \gamma$ is the m-th harmonic number: doubling the number of changes adds only about $(\ln 2)/p$ rounds.

Corollary 4.6 (Rounds with confidence)**.** *For every $\eta \in (0, 1)$, the task ends within*

$$r_\eta = \left\lceil \frac{\ln(m/\eta)}{-\ln(1-p)} \right\rceil \tag{4.12}$$

rounds with probability at least $1 - \eta$.

Proof. By the union bound, $\mathbb{P}(R_{\max} > r) \le m\,\mathbb{P}(R > r) = m(1-p)^r$, which is at most η as soon as $r \ge \ln(m/\eta)/(-\ln(1-p))$. □

Example 4.7. With $p = 0.8$ and $m = 5$, eq. (4.11) gives $\mathbb{E}[R_{\max}] \approx 1.91$ and eq. (4.12) with $\eta = 0.05$ gives $r_\eta = 3$: most tasks finish in one or two rounds, and almost all within three.

5 Concurrent tasks and conflicts

Papilot runs up to $c = 2$ tasks at once, each against the manuscript as it was when the task started. Accepted text from one task can therefore overlap a later proposal from another, and eq. (4.4) then returns $\perp$. This section estimates how often that happens.

5.1 Uniformly placed edits

Assumption 5.1. Each task proposes edits of a common length ℓ whose start positions are independent and uniformly distributed over a document of length $n \gg \ell$.

Proposition 5.2 (Conflict probability)**.** *Under assumption 5.1, let two tasks propose k_1 and k_2 edits. The number X of incompatible cross pairs has mean*

$$\mathbb{E}[X] = \frac{(2\ell + 1)\, k_1 k_2}{n}, \tag{5.1}$$

and the probability that the tasks conflict at all is

$$\mathbb{P}(X > 0) = 1 - \Big(1 - \frac{2\ell + 1}{n}\Big)^{k_1 k_2} \approx 1 - \exp\Big(-\frac{(2\ell + 1)\, k_1 k_2}{n}\Big). \tag{5.2}$$

Proof. Two edits of length ℓ starting at s and s' violate eq. (4.2) exactly when $|s - s'| \le \ell$, an event of probability $(2\ell + 1)/n$ up to boundary effects of order ℓ/n. Summing over the $k_1 k_2$ cross pairs gives eq. (5.1). Treating the pairs as independent gives the product in eq. (5.2), and $\log(1-x) = -x + O(x^2)$ gives the approximation, which is the Poisson approximation $\mathbb{P}(X = 0) \approx e^{-\mathbb{E}[X]}$ [6]. □

For a 40-page paper with $n \approx 1.2 \times 10^5$ characters, two tasks of $k_1 = k_2 = 10$ edits of $\ell = 200$ characters have $\mathbb{E}[X] \approx 0.33$ and conflict with probability about $1 - e^{-0.33} \approx 0.28$.

5.2 How much overlaps

When two edits do conflict, the overlap is usually partial.

Proposition 5.3 (Expected overlap)**.** *Conditioned on a conflict, the offset $d = s' - s$ is uniform on $\{-\ell, \dots, \ell\}$ and the overlap $\ell - |d|$ has mean*

$$\mathbb{E}\big[\ell - |d| \;\big|\; |d| \le \ell\big] = \frac{1}{2\ell + 1} \sum_{d=-\ell}^{\ell} (\ell - |d|) = \frac{\ell^2}{2\ell + 1} \approx \frac{\ell}{2}. \tag{5.3}$$

Proof. The sum is $\ell + 2\sum_{j=1}^{\ell}(\ell - j) = \ell + \ell(\ell - 1) = \ell^2$. □

A typical conflict therefore involves about half of an edit, which matches the experience that conflicting proposals usually touch the same sentence rather than the same word.

5.3 Clustered edits

Real revisions are not uniform: an author working on the introduction asks for several changes there. We model this by letting both tasks place their edits uniformly within a common region covering a fraction $\nu \in (0, 1]$ of the document.

Corollary 5.4 (Clustering raises conflicts)**.** *If both tasks edit only within a region of length νn,*

$$\mathbb{P}(X > 0) \approx 1 - \exp\Big(-\frac{(2\ell + 1)\, k_1 k_2}{\nu n}\Big), \tag{5.4}$$

so that confining two tasks to a tenth of the paper multiplies the expected number of conflicts by ten.

Proof. Apply proposition 5.2 with n replaced by νn. □

Corollary 5.4 argues for fewer, longer tasks over many short ones that touch the same part of a paper, and for finishing the review of one task before starting another in the same section. It also explains why Papilot surfaces conflicts from Git rather than resolving them: a resolution would have to choose between two texts the author has not yet compared.

6 Building the PDF

6.1 Snapshots of the working tree

The PDF view shows the manuscript exactly as it is on disk, including accepted but uncommitted changes. Let W be the working tree and H the tree of `HEAD`. A temporary index I records the working tree without touching the real index:

$$I \leftarrow \text{read-tree}(H), \qquad I \leftarrow I \cup \text{add}(W), \qquad T = \text{write-tree}(I). \tag{6.1}$$

The tree T is a content hash of every tracked and untracked, non-ignored file. The build is cached under the key

$$\text{key}(W) = h\big(T \,\|\, \text{build settings}\big), \tag{6.2}$$

where h is a cryptographic hash and the settings name the engine and the main file.

Proposition 6.1 (Undo never recompiles)**.** *Let W_0 be a working tree whose build has completed, and let a sequence of decisions followed by their undos return the files to the same content. Then the final working tree has the key of W_0 and its PDF is served from the cache.*

Proof. T in eq. (6.1) depends only on file contents, so equal contents give equal trees and, by eq. (6.2), equal keys. □

6.2 When to build

While the PDF view is open, Papilot checks the working tree every second and a half during a build, every fifteen seconds otherwise, and whenever the window regains focus. Let checks happen at rate at least φ, edits at rate μ and builds take time β. A build that is running when the tree changes is not cancelled; the first check after it finishes starts a build for the newer key. An edit made just after a build started waits for that build, then for the next check, then for its own build, so the PDF is never older than

$$\text{staleness} \le 2\beta + \frac{1}{\varphi}, \tag{6.3}$$

and the number of builds per unit time is at most $\min\big(\mu, \varphi, 1/\beta\big)$ in the long run: frequent edits do not cause a backlog of builds, they only skip intermediate states.

6.3 Reviewing changed pages

A review compares the PDF of the round's base with the PDF after the task's edits. Let the two documents have pages $P = (p_1, \dots, p_N)$ and $P' = (p'_1, \dots, p'_M)$ with content hashes $h(p_i)$. Inserting a paragraph shifts every later page, so comparing pages by index would mark almost everything as changed. A robust comparison aligns the pages by a longest common subsequence of their hashes,

$$\mathrm{LCS}(P, P') = \max\Big\{ |S| : S \subseteq [N] \times [M] \text{ increasing},\ h(p_i) = h(p'_j)\ \forall (i,j) \in S \Big\}, \tag{6.4}$$

computed in $O((N+M)D)$ time, where $D = N + M - 2|S|$ is the size of the edit script [16]. Papilot's review goes further and renders only the strips of each page around the changed source lines, so that a one-word change is reviewed as one or two lines of typeset text.

7 Geometry of the PDF view

7.1 From a glyph to a pixel

A PDF page lives in user space, in points ($1\,\mathrm{bp} = 1/72\,\mathrm{in}$), with the origin at the bottom left of its view box $[x_0, y_0, x_1, y_1]$. Following the PDF convention we use row vectors. A glyph's origin is placed by the text rendering matrix of eq. (3.2),

$$\begin{pmatrix} x & y & 1 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 1 \end{pmatrix} T_{rm}, \tag{7.1}$$

and the viewer draws the page at scale s (device pixels per point) with the origin at the top left, so for an unrotated page the viewport matrix is

$$V_s = \begin{pmatrix} s & 0 & 0 \\ 0 & -s & 0 \\ -s x_0 & s y_1 & 1 \end{pmatrix}, \qquad \begin{pmatrix} u & v & 1 \end{pmatrix} = \begin{pmatrix} x & y & 1 \end{pmatrix} V_s. \tag{7.2}$$

A page rotated by $\phi \in \{0, \frac{\pi}{2}, \pi, \frac{3\pi}{2}\}$ is drawn with V_s composed with a rotation about the page centre,

$$R_\phi = \begin{pmatrix} \cos\phi & \sin\phi & 0 \\ -\sin\phi & \cos\phi & 0 \\ 0 & 0 & 1 \end{pmatrix}, \tag{7.3}$$

so that the complete chain from a glyph to a pixel is $(0\ 0\ 1)\, T_{rm}\, V_s\, R_\phi$ up to a translation. Zooming changes only s, which is why Papilot stores positions on the page in normalised coordinates $r = (\xi_0, \eta_0, \xi_1, \eta_1) \in [0,1]^4$ relative to the view box:

$$\xi = \frac{x - x_0}{x_1 - x_0}, \qquad \eta = \frac{y_1 - y}{y_1 - y_0}. \tag{7.4}$$

7.2 How TEX spaces a line

TEX sets a line of natural width L_0 in a box of width L by stretching or shrinking the glue between words [10]. If the line's glue has total stretchability Y and shrinkability Z, the *adjustment ratio* is

$$r = \begin{cases} (L - L_0)/Y, & L \geq L_0, \\ (L - L_0)/Z, & L < L_0, \end{cases} \tag{7.5}$$

and an interword space with natural width g, stretch y and shrink z is set at width $g + ry$ or $g + rz$. The *badness* of the line is approximately

$$b(r) = \min\bigl(10000,\ 100\,|r|^3\bigr), \tag{7.6}$$

with 10000 meaning infinitely bad. The Knuth–Plass algorithm chooses breaks that minimise the total *demerits* [11], where a line with badness b ending at a break with penalty p costs

$$d = \begin{cases} (l+b)^2 + p^2, & 0 \le p < 10000, \\ (l+b)^2 - p^2, & -10000 < p < 0, \\ (l+b)^2, & p \le -10000, \end{cases} \tag{7.7}$$

and l is the line penalty (ten by default). Because eq. (7.7) trades badness between lines, interword spaces vary from line to line, and that variation is what a text layer must follow.

7.3 Why a line-wide text layer drifts

To make text selectable, a viewer overlays invisible text on the rendered page. A common construction places one run per line in a stand-in font and stretches it horizontally so that the run's ends meet the line's ends [15]. Let a line consist of glyphs with true advances $a_1, \dots, a_N$ and stand-in advances $b_1, \dots, b_N$. The stretch factor is

$$\sigma = \frac{\sum_{i=1}^{N} a_i}{\sum_{i=1}^{N} b_i}, \tag{7.8}$$

and the selectable position of glyph k is off by

$$\varepsilon_k = \sum_{i<k} \delta_i, \qquad \delta_i = \sigma b_i - a_i, \qquad \varepsilon_1 = \varepsilon_{N+1} = 0. \tag{7.9}$$

Theorem 7.1 (Mid-line drift)**.** *If the discrepancies $\delta_1, \dots, \delta_N$ are exchangeable with variance v, then $\sum_i \delta_i = 0$ forces $\operatorname{Cov}(\delta_i, \delta_j) = -v/(N-1)$ for $i \neq j$, and*

$$\operatorname{Var}(\varepsilon_k) = v\,\frac{(k-1)(N-k+1)}{N-1}, \qquad \max_k \operatorname{Var}(\varepsilon_k) \approx \frac{vN}{4}. \tag{7.10}$$

The proof is in section A. Equation (7.10) is the variance of a discrete Brownian bridge [2]: the error vanishes at both ends of the line, where the run is anchored, and is largest in the middle.

Justified lines add a deterministic bias. Every space is printed $g + ry$ wide but drawn σb_{space} wide in the stand-in font, so that

$$\beta_k = \bigl(g + ry - \sigma b_{\text{space}}\bigr) \cdot \#\{\text{spaces before glyph } k\}. \tag{7.11}$$

With TEX's interword glue at ten points, $g = 3.33\,\text{pt}$ and $y = 1.67\,\text{pt}$, a line set with $r = 0.6$ has spaces of 4.33 pt; if the stand-in font's spaces come out at 3.0 pt after stretching, the bias grows by 1.33 pt per space and reaches 8 pt, more than a letter and a half, after six words. Together, eqs. (7.10) and (7.11) explain drifts of several characters in the middle of a line, enough for a drag to select a neighbouring word.

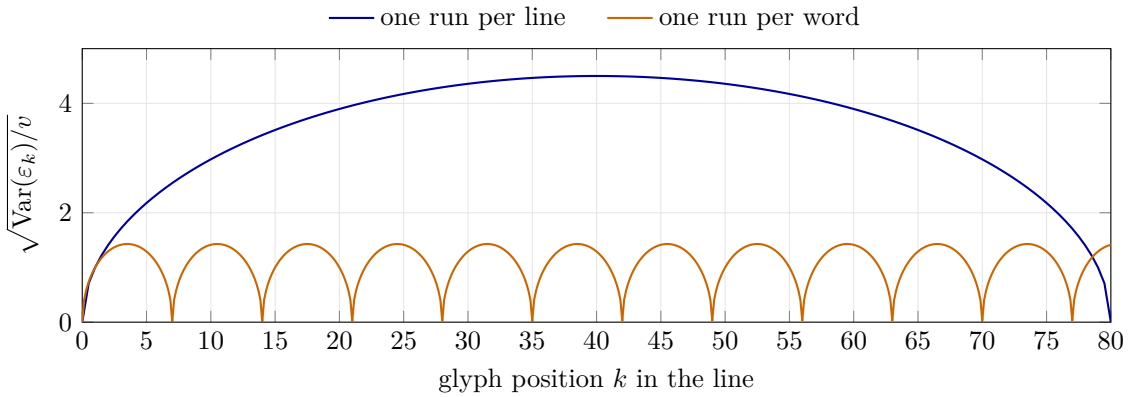


Figure 1: Standard deviation of the selection error along a line of 80 glyphs, from eq. (7.10), for a single stretched run and for per-word runs with words of six letters and a space. The per-word error returns to zero at every word.

7.4 Per-word runs

Papilot instead reads every word's printed place from the PDF: its left edge λ_j, baseline y_j, size f_j and width w_j. It places one run per word, stretched to end where the next word begins, so that the run for word j covers glyphs J_j and the space after them with its own factor

$$\sigma_j = \frac{\lambda_{j+1} - \lambda_j}{\sum_{i \in J_j} b_i + b_{\text{space}}}. \tag{7.12}$$

Theorem 7.2 (Per-word drift)**.** *With per-word runs, every word starts exactly where it is printed, and the error at a glyph k of word j satisfies*

$$|\varepsilon_k| \le \sum_{i \in J_j} |\sigma_j b_i - a_i| \le N_j \max_{i \in J_j} |\sigma_j b_i - a_i|, \qquad \mathrm{Var}(\varepsilon_k) \le \frac{v N_j}{4}, \tag{7.13}$$

where $N_j = |J_j|$. The bias eq. (7.11) does not accumulate across words.

Proof. The run for word j starts at λ_j, so ε is zero at the first glyph of every word. Within the word, eq. (7.9) applies with N replaced by N_j and σ by σ_j, which gives the first two bounds by the triangle inequality and the third by theorem 7.1. Since every word restarts at its printed position, no space contributes to the error of a later word. □

For a line of $N = 80$ glyphs and words of $N_j \approx 6$, the typical error falls by a factor of about $\sqrt{N/N_j} \approx 3.7$ and the bias disappears. Figure 1 compares the two layouts.

8 Notes on the PDF

An author can also ask for changes directly on the PDF: select text, or mark a region around a figure or an equation, and write a note. All pending notes go to the agent as one task. For that task to be useful, each note must name the source it refers to.

Algorithm 1 Settling one end of a selection

Require: printed line p, index α of the marked letter in $\lambda(p)$, source string S with line map π

for $\rho \in (\infty, 24, 10)$ **do** ▷ whole line first, then shorter context
  $o_0 \leftarrow \max(0, \alpha - \rho)$
  $q \leftarrow \lambda(p)_{[o_0,\ \alpha+\rho]}$
  $O \leftarrow \{o : S_{[o,\, o+|q|)} = q\}$
  if $O \neq \emptyset$ **then**
    $o^\star \leftarrow \arg\min_{o \in O} \operatorname{dist}(\pi(o + \alpha - o_0))$
    return $\pi(o^\star + \alpha - o_0)$
return SyncTeX's line ▷ formulas and other unmatched text

8.1 From a mark to source lines

A note is a set of rectangles $\{r_1, \ldots, r_q\}$ in the normalised coordinates of eq. (7.4) on page P, together with a comment. For every printed line the selection touches, Papilot asks SyncTeX for the source position of the first and the last marked character [13]:

$$\mathcal{S}(P, x, y) = (\text{file}, \text{line}), \qquad L = \{\, \mathcal{S}(P, c) : c \in C_{\text{first}} \cup C_{\text{last}} \,\}, \qquad [\ell_-, \ell_+] = [\min L, \max L]. \quad (8.1)$$

SyncTeX records boxes and glue rather than characters. The space before a word that begins a printed line comes from the line break of the *previous* source line, so such a word is credited to that line. Papilot therefore *settles* each end of the selection by matching its printed context against the source near SyncTeX's answer.

8.2 Settling a selection

Let $\lambda(\cdot)$ map text to its lowercase letters and digits, after Unicode compatibility normalisation, with TEX control sequences and comments removed from the source. For a source window $[\ell_- - 4, \ell_+ + 4]$ we form the string $S = \lambda(\text{lines})$ and remember for each position the line it came from, $\pi : \{1, \ldots, |S|\} \to \mathbb{N}$. The distance of a line from SyncTeX's range is

$$\operatorname{dist}(\ell) = \max(0,\ \ell_- - \ell,\ \ell - \ell_+). \quad (8.2)$$

Algorithm 1 runs once for the first and once for the last marked character. If the settled range is empty or inverted, Papilot keeps SyncTeX's range. Because the comparison uses letters only, subscripts, macros and hyphenation do not prevent a match; formulas, whose printed form shares no letters with their source, fall back to SyncTeX by design.

8.3 When is a match unique?

Algorithm 1 picks the occurrence nearest to SyncTeX's answer, so a spurious occurrence matters only if it is closer than the true one. Model the letters of the source as a stationary source with entropy $\mathcal{H}$ bits per letter. The expected number of chance occurrences of a context of $|q|$ letters in a window of $|S|$ letters is about

$$\mathbb{E}[\text{spurious}] \approx |S| \cdot 2^{-\mathcal{H}|q|}. \quad (8.3)$$

Estimates of the entropy of English range from about one to four bits per letter depending on how much context is used [17]. Even with the pessimistic $\mathcal{H} = 1$, a context of $|q| = 21$ letters (the shortest one algorithm 1 tries) in a window of $|S| = 2000$ letters gives $\mathbb{E}[\text{spurious}] \approx 2000 \cdot 2^{-21} \approx 10^{-3}$. Spurious matches come instead from genuine repetition, such as a phrase used twice in nearby paragraphs, which is exactly the case the distance eq. (8.2) resolves.

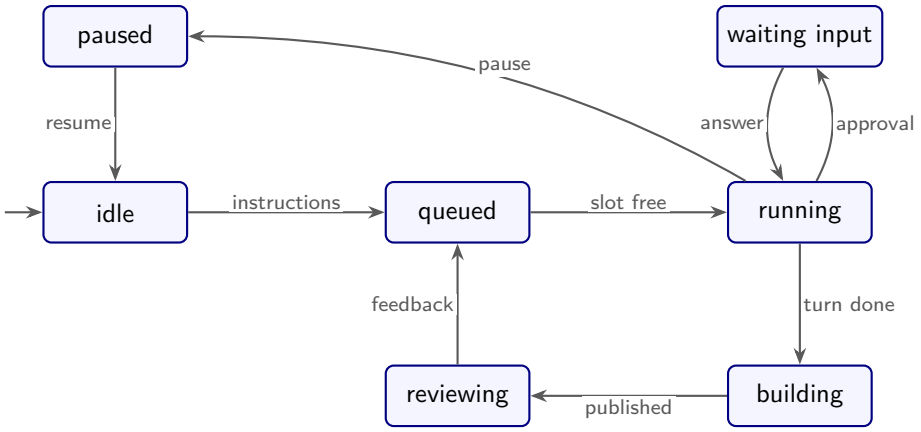


Figure 2: Task states. A turn that fails or whose outcome is uncertain leaves the task in a recoverable state with a *Check status* action (not shown).

8.4 Selecting whole words

Let a drag select characters $[s, t)$ of the per-word text layer of theorem 7.2. Papilot widens the selection to whole words, but only at a boundary that cuts through a word:

$$s' = \begin{cases} \max\{i \le s : i = 0 \text{ or } c_{i-1} \text{ is not a letter}\}, & c_s \text{ is a letter}, \\ s, & \text{otherwise}, \end{cases} \tag{8.4}$$

and symmetrically for t. A drag that ends at the start of the next word, as browsers often report, therefore keeps exactly the words it covers. The note stores the widened quote, one rectangle per printed line, and the source lines of algorithm 1 together with an excerpt of their LaTeX, so that the agent receives formulas as source rather than as extracted text.

9 Agents and scheduling

9.1 One protocol for every agent

Every agent is driven through the same small protocol: a thread is started or resumed, a turn is started with the author's text, the agent streams items (messages, commands, file changes, questions), and the turn completes, is interrupted, or fails. Formally a task moves through the states of fig. 2.

Codex speaks this protocol natively. Claude runs through the Claude Agent SDK, Antigravity runs the `agy` command line once per turn, and Kimi Code runs `kimi acp`; adapters translate their interfaces into the same requests and notifications. Kimi sessions resume after a restart. Each agent uses its own login on the host, and Papilot never reads, stores or forwards a credential. An agent that cannot ask for permission, such as a headless command line, is told in advance which tools it may not use.

9.2 Effort and cost

A reasoning effort ϵ from the ordered set

$$\mathcal{E} = \{\mathsf{low} < \mathsf{medium} < \mathsf{high} < \mathsf{xhigh} < \mathsf{max}\} \tag{9.1}$$

is chosen per task and remembered per agent. If a turn at effort ϵ consumes on average $\kappa(\epsilon)$ tokens and produces an accepted change with probability $q(\epsilon)$, the number of turns until acceptance is geometric and the expected cost of an accepted change is

$$\mathrm{cost}(\epsilon) = \frac{\kappa(\epsilon)}{q(\epsilon)} \cdot \mathrm{price}, \qquad \epsilon^\star = \arg\min_{\epsilon \in \mathcal{E}} \mathrm{cost}(\epsilon). \tag{9.2}$$

A lower effort is cheaper only if $\kappa(\epsilon)/\kappa(\epsilon') < q(\epsilon)/q(\epsilon')$: a failed turn must be paid for again, and so must the author's review of it.

9.3 Queueing

Tasks arrive at rate ν and occupy a slot for a mean time $1/\omega$. With c slots, exponential times and offered load $A = \nu/\omega < c$, Papilot is an $M/M/c$ queue [9]. The probability that a new task waits is given by the Erlang C formula [5]:

$$C(c, A) = \frac{\dfrac{A^c}{c!}\,\dfrac{c}{c - A}}{\displaystyle\sum_{k=0}^{c-1} \frac{A^k}{k!} + \frac{A^c}{c!}\,\frac{c}{c - A}}, \qquad \mathbb{E}[\mathrm{wait}] = \frac{C(c, A)}{c\,\omega - \nu}. \tag{9.3}$$

For $c = 2$ this simplifies to $C(2, A) = A^2/(2{+}A)$. An author who starts a task every ten minutes, each taking six minutes, has $A = 0.6$ and waits with probability about 0.14, for $0.14/(2/6 - 1/10) \approx 0.6$ minutes on average. By Little's law [14], the mean number of tasks in the system is

$$\bar{N} = \nu\Big(\mathbb{E}[\mathrm{wait}] + \frac{1}{\omega}\Big). \tag{9.4}$$

9.4 Approvals

A turn that asks for approval stops until the author answers. If approvals arrive at rate ζ during a turn of working time T_w and the author answers after a mean delay Θ, the expected wall-clock time of the turn is

$$\mathbb{E}[T] = T_w(1 + \zeta\Theta), \tag{9.5}$$

which motivates reducing unnecessary approval requests. Codex and Claude can run sandboxed commands under their permission policies; Kimi uses manual approvals for tools on the host, without a Papilot sandbox. Antigravity edits files but cannot run shell commands in Papilot.

10 Usage windows

10.1 Utilisation

Agent subscriptions meter usage over rolling windows, typically a five-hour window and a weekly one. If $r(\tau)$ is the rate at which a plan's allowance is consumed, the utilisation of a window of length w at time t is

$$u_w(t) = \frac{1}{B_w} \int_{t-w}^{t} r(\tau)\, d\tau \in [0, 1], \tag{10.1}$$

where B_w is the window's budget. Papilot shows u_w for each window the agent reports, coloured by thresholds

$$\text{colour}(u) = \begin{cases} \text{blue}, & u < 0.75, \\ \text{amber}, & 0.75 \le u < 0.9, \\ \text{red}, & u \ge 0.9 \text{ or the limit is reached}, \end{cases} \tag{10.2}$$

together with the reset time: as a countdown when it is less than a day away, and as a day and time otherwise.

10.2 Will the limit be reached?

At a constant rate $\bar{r}$, the time left before the limit is

$$t_{\text{left}} = \frac{(1 - u_w(t))B_w}{\bar{r}}; \tag{10.3}$$

with the weekly window at $u = 0.83$ and an even pace over the week, $\bar{r} = B_w/7$ days and about 1.2 days of work remain.

Usage is lumpy, however: it arrives in turns. Model turns as a Poisson process of rate λ with independent costs J of mean μ_J and variance s_J^2. The usage $S(t)$ over the next t is compound Poisson, with

$$\mathbb{E}[S(t)] = \lambda t \mu_J, \qquad \text{Var}(S(t)) = \lambda t(\mu_J^2 + s_J^2) \tag{10.4}$$

by Wald's identity and the law of total variance [6]. With t_r the time until the window resets, the normal approximation gives the probability of reaching the limit before the reset:

$$\mathbb{P}\big(S(t_r) > (1-u)B_w\big) \approx 1 - \Phi\left(\frac{(1-u)B_w - \lambda t_r \mu_J}{\sqrt{\lambda t_r(\mu_J^2 + s_J^2)}}\right), \tag{10.5}$$

where Φ is the standard normal distribution function. Large, rare turns (high s_J) make the limit more likely than the mean alone suggests, which is one more reason to prefer a lower effort for routine edits.

11 Closing the loop with Git

Accepted changes land in the working tree without a commit, so that the author decides what history to write. The Git panel shows the branch and how it compares with its upstream, the last commit, and the uncommitted changes with their diffs; from it the author commits chosen files, fetches, pulls and pushes.

11.1 Committing chosen files

Write a tree as a map from paths to contents and, for a set of paths P, let $X|_P$ be the restriction of X to P and $\overline{P}$ the complement. Let HEAD have tree H, let I be the index and W the working tree, and let P be the set of paths the author ticks.

Proposition 11.1 (Partial commits)**.** *Committing P produces the tree and leaves the index as*

$$T_c = H|_{\overline{P}} \cup W|_P, \qquad I' = I|_{\overline{P}} \cup W|_P, \tag{11.1}$$

that is, exactly the chosen files as they are on disk are committed, while anything staged for other paths stays staged and out of the commit. A rename from π to π' contributes both π and π' to P.

The proof, which follows the semantics of `git add` followed by `git commit -only`, is in section A. Paths are passed to Git literally, so that a file name cannot act as a pattern, and new LaTeX build output (`.aux`, `.log` and similar files) starts unticked.

11.2 Synchronising with the remote

Let $\mathrm{anc}(X)$ be the set of commits reachable from X, including X. With local head L and upstream U, the branch is ahead and behind by

$$\text{ahead} = |\mathrm{anc}(L) \setminus \mathrm{anc}(U)|, \qquad \text{behind} = |\mathrm{anc}(U) \setminus \mathrm{anc}(L)|, \tag{11.2}$$

the two numbers shown beside the branch name. Pull is a fast-forward only. It succeeds exactly when

$$\mathrm{anc}(L) \subseteq \mathrm{anc}(U) \iff \text{merge-base}(L, U) = L \iff \text{ahead} = 0, \tag{11.3}$$

in which case $L \leftarrow U$; otherwise Papilot reports the divergence and never merges or rebases on the author's behalf. Push is the mirror image and succeeds when behind $= 0$. A branch without an upstream is published with its first push.

Remark 11.2. Fetch, pull and push use the host's own Git credentials with prompts disabled and a time limit, so that a missing credential produces Git's own error message instead of a request that hangs.

12 Security

Papilot runs on the author's own computer and can be reached from other devices over a private network. Its threat model is a web page in another tab that tries to act on the author's behalf, and a device on the network that is not the author's.

12.1 Requests

Every state-changing request must carry a token χ of 256 random bits, issued with the page and compared in constant time, which defends against cross-site request forgery [1]. A remote request must also come from the configured origin and, over the private network, from the configured owner. Writing ρ for a request,

$$\mathrm{allow}(\rho) = (\mathrm{safe}(\rho) \vee \mathrm{token}(\rho) = \chi) \wedge \mathrm{origin}(\rho) \in \mathcal{O} \wedge \mathrm{owner}(\rho) = o, \tag{12.1}$$

where $\mathrm{safe}(\rho)$ holds for requests that only read. A guess succeeds with probability 2^{-256}, and a constant-time comparison leaks no prefix of χ through timing.

12.2 Agents

Agents run with the author's own logins but within limits Papilot sets:

- each task's agent works in its own worktree w_i and is told never to edit the manuscript checkout w_0, other worktrees, or Papilot itself;
- commands run in the agent's sandbox, and anything that leaves it is either brought to the author for approval or, for an agent that cannot ask, declined in advance;
- only the author accepts changes into w_0, through the review of section 4.

The last point is the one that matters most: even an agent that ignored its instructions could not change the manuscript without a decision the author made while looking at the typeset result.

Table 1: Expected rounds $\mathbb{E}[R_{\max}]$ from eq. (4.11), and in parentheses the rounds $r_{0.05}$ within which a task ends with probability at least 0.95 from eq. (4.12).

	changes per task m				
acceptance p	1	2	5	10	20
0.5	2.00 (5)	2.67 (6)	3.79 (7)	4.73 (8)	5.69 (9)
0.7	1.43 (3)	1.76 (4)	2.39 (4)	2.93 (5)	3.49 (5)
0.9	1.11 (2)	1.21 (2)	1.46 (2)	1.76 (3)	2.08 (3)

Table 2: Probability that two concurrent tasks with k edits each conflict, from eq. (5.2) with $\ell = 200$.

	edits per task k		
source length n	5	10	20
3.0×10^4 (short paper)	0.284	0.737	0.995
1.2×10^5 (long paper)	0.080	0.284	0.737
4.8×10^5 (thesis)	0.021	0.080	0.284

13 A model-based evaluation

This section evaluates the models of the previous sections numerically. Every number below is computed from a formula in this paper; none is measured. The purpose is to show what the formulas imply for realistic parameters, not to report an empirical study.

13.1 Rounds per task

Table 1 evaluates proposition 4.5 and corollary 4.6 for a range of acceptance probabilities and numbers of changes. Two patterns stand out. The expected number of rounds grows only logarithmically with the number of changes, as the harmonic-number approximation predicts. And the 95 % bound $r_{0.05}$ is at most two and a half times the mean in every case shown, so that at $p = 0.7$ a task with five changes that is still open after four rounds is among the slowest five per cent, and worth rewriting rather than reviewing again.

13.2 Conflicts

Table 2 evaluates eq. (5.2) for two concurrent tasks with edits of $\ell = 200$ characters. A 10-page paper has about 3×10^4 characters of source, a 40-page paper about 1.2×10^5 and a thesis about 4.8×10^5. By corollary 5.4, confining both tasks to a quarter of a document moves a row up one line.

13.3 Waiting for a slot

Figure 3 plots the waiting probability eq. (9.3) against the offered load for one, two and three slots. At the load $A = 0.6$ of the example in section 9, a second slot lowers the probability of waiting from 0.6 to 0.14, and a third would lower it to 0.025. Two slots are a good compromise between waiting and the conflicts of table 2, which grow with the number of tasks that run at once.

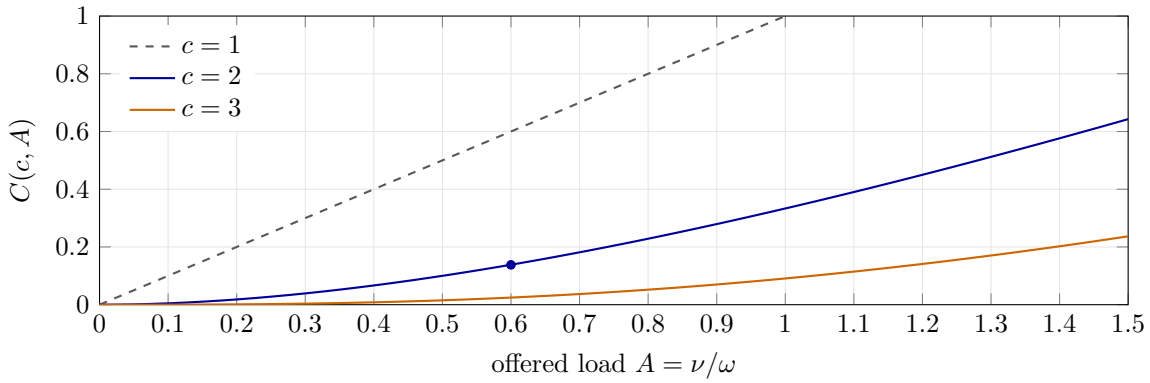


Figure 3: Probability that a new task waits for a slot, from the Erlang C formula eq. (9.3). The dot marks the example of section 9.

13.4 Selection error

For the text layer, fig. 1 already compares the two layouts. In numbers: with $v = (0.4\,\mathrm{pt})^2$ per glyph, a line of 80 glyphs has a mid-line standard deviation of $0.4\sqrt{80/4} \approx 1.8\,\mathrm{pt}$ from eq. (7.10) alone, plus the bias of eq. (7.11), which reaches several points; per-word runs of seven glyphs bound it by $0.4\sqrt{7/4} \approx 0.53\,\mathrm{pt}$, well under the width of one letter.

14 Related work

Differences between documents. Line-based differences go back to Hunt and McIlroy [7] and Myers's $O(ND)$ algorithm [16], which Git uses by default. `latexdiff` [19] marks up the differences between two LaTeX sources so that they can be typeset together. Papilot instead typesets each version separately and shows the corresponding strips of the two PDFs one above the other, which keeps the typeset result of each version exact at the cost of a second build.

Concurrent editing. Operational transformation [4] and conflict-free replicated data types [18] let several writers edit a shared document and converge without coordination. Papilot's setting is different: proposals are not applied until an author accepts them, so convergence is a property of accepted edits only, and lemma 4.4 needs to hold only for compatible pairs. Incompatible pairs are surfaced to the author rather than transformed.

Line breaking and text layers. The Knuth–Plass algorithm [11] is the reason LaTeX spaces vary from line to line, and with it the drift analysed in section 7. PDF viewers such as PDF.js [15] place an invisible text layer over the rendered page; Papilot uses PDF.js for rendering, search and links, and replaces only the placement of its text with the per-word runs of theorem 7.2.

Source synchronisation. SyncTeX [13] maps between typeset positions and source lines and underlies forward and inverse search in most TeX editors. Papilot uses its reverse queries and corrects their line-boundary error with algorithm 1.

Queueing. The waiting-time results of section 9 are classical; we use Erlang's formula [5], Little's law [14] and the treatment of $M/M/c$ queues in Kleinrock [9].

15 Discussion and conclusion

15.1 Limitations

Three limitations are worth stating. First, proposition 5.2 assumes uniformly placed edits; real revisions cluster around the parts of a paper an author is working on, which by corollary 5.4 raises the conflict rate and argues for fewer, longer tasks over many short ones. Second, algorithm 1 cannot settle formulas, since their printed form shares no letters with their source; for those, SyncTeX's lines are kept and the excerpt shows the whole display. Third, an agent that cannot ask for permission, such as a headless command line, must be restricted in advance; Papilot runs such agents so that they edit only the task's worktree, which also means they cannot compile or test their own changes.

15.2 Conclusion

Papilot treats agent editing as a sequence of reviewable rounds. Isolation makes proposals safe, typeset review makes them legible, and explicit acceptance keeps the author in charge of the paper. The models in this paper make those properties checkable: compatible edits commute and rebase (proposition 4.3 and lemma 4.4), tasks need few rounds (proposition 4.5 and corollary 4.6), undo never recompiles (proposition 6.1), per-word text bounds the selection error (theorem 7.2), and partial commits keep unrelated staged work intact (proposition 11.1). They also give the reader something worth reviewing.

A Proofs

A.1 Expected rounds

We prove proposition 4.5. Write $q = 1 - p$. The rounds of different changes are independent, so $\mathbb{P}(R_{\max} \le r) = \mathbb{P}(R \le r)^m = (1 - q^r)^m$, and since $R_{\max}$ takes values in $\mathbb{N}$,

$$\mathbb{E}[R_{\max}] = \sum_{r \ge 0} \mathbb{P}(R_{\max} > r) = \sum_{r \ge 0} \Big(1 - \big(1 - q^r\big)^m\Big). \tag{A.1}$$

By the binomial theorem, $1 - (1 - q^r)^m = \sum_{k=1}^{m} (-1)^{k+1} \binom{m}{k} q^{rk}$. The inner sum is finite, so the two sums may be exchanged, and $\sum_{r \ge 0} q^{rk} = 1/(1 - q^k)$ gives eq. (4.11).

For the asymptotics, $1 - q^k = 1 - (1 - p)^k = kp + O(p^2)$ as $p \to 0$, so

$$\mathbb{E}[R_{\max}] = \frac{1}{p} \sum_{k=1}^{m} \frac{(-1)^{k+1}}{k} \binom{m}{k} + O(1) = \frac{H_m}{p} + O(1), \tag{A.2}$$

using the identity $\sum_{k=1}^{m} \frac{(-1)^{k+1}}{k} \binom{m}{k} = H_m$, which follows by integrating $(1 - (1 - x)^m)/x = \sum_{j=0}^{m-1} (1 - x)^j$ from 0 to 1. □

A.2 Mid-line drift

We prove theorem 7.1. Let $\gamma = \operatorname{Cov}(\delta_i, \delta_j)$ for $i \ne j$, which does not depend on the pair by exchangeability. Since $\sum_{i=1}^{N} \delta_i = 0$,

$$0 = \operatorname{Var}\Big(\sum_{i=1}^{N} \delta_i\Big) = Nv + N(N-1)\,\gamma, \qquad \text{so} \qquad \gamma = -\frac{v}{N-1}. \tag{A.3}$$

With $j = k - 1$ terms in ε_k,

$$\operatorname{Var}(\varepsilon_k) = jv + j(j-1)\,\gamma = jv\Big(1 - \frac{j-1}{N-1}\Big) = v\,\frac{j\,(N-j)}{N-1} = v\,\frac{(k-1)(N-k+1)}{N-1}. \tag{A.4}$$

The product $j(N-j)$ is largest at $j = N/2$, where it equals $N^2/4$, and $N^2/(4(N-1)) \approx N/4$ for large N. □

A.3 Partial commits

We prove proposition 11.1. Papilot commits P in two steps. First, adding P to the index sets $I(\pi) = W(\pi)$ for every $\pi \in P$ that exists on disk and removes from the index every $\pi \in P$ that does not, leaving the entries for $\overline{P}$ unchanged:

$$I_1 = I|_{\overline{P}} \cup W|_P. \tag{A.5}$$

Second, a commit restricted to P builds its tree from H with the entries for P taken from the working tree and ignores the entries of the index for $\overline{P}$, which gives $T_c = H|_{\overline{P}} \cup W|_P$. It then updates the index entries for P and keeps the others, so $I' = I_1$. For a rename from π to π', the entry for π is absent from W; including π in P removes it from T_c, and including π' adds the new path. □

B Notation and keyboard

B.1 Notation

Symbol	Meaning	Where
x_k, F	auxiliary state of the build, one run	eq. (3.1)
T_{rm}, T_m	text rendering and text matrices	eq. (3.2)
D, n	manuscript source and its length	section 4
$e = (a, b, w)$, $\delta(e)$	edit replacing $[a, b)$ by w, its shift	eq. (4.1)
$e \perp e'$	compatible edits	eq. (4.2)
$\tau(e', e)$	e' rebased past e	eq. (4.4)
θ, α	decision state, action	eqs. (4.7) and (4.8)
p, $R_{\max}$, r_η	acceptance probability, rounds, rounds with confidence	eqs. (4.11) and (4.12)
ℓ, k_i, ν	edit length, edits per task, clustered fraction	eqs. (5.2) and (5.4)
T, key(W)	snapshot tree, build key	eqs. (6.1) and (6.2)
s, V_s, R_ϕ	viewer scale, viewport, rotation	eq. (7.2)
r, $b(r)$, d	adjustment ratio, badness, demerits	eqs. (7.5) to (7.7)
ε_k, σ, β_k	text layer drift, stretch factor, bias	eqs. (7.9) and (7.11)
$\mathcal{S}$, λ, dist	SyncTeX query, letter normalisation, line distance	section 8
ϵ, κ, q	effort, tokens per turn, acceptance per turn	eq. (9.2)
c, A, $C(c, A)$	slots, offered load, waiting probability	eq. (9.3)
u_w, B_w, $S(t)$	window utilisation, budget, usage process	eqs. (10.1) and (10.4)
H, I, W, P	Git trees, index, working tree, chosen paths	eq. (11.1)
anc(X)	commits reachable from X	eq. (11.2)
χ, $\mathcal{O}$, o	request token, allowed origins, owner	eq. (12.1)

B.2 Keyboard

Key	Action
`A`, `R`, `C`	accept, reject or comment on the current change
`J`, `K`	next and previous change
`N`	next undecided change
`S`	show the LaTeX source of the current change
`Cmd` + `F`	find in the PDF
`Cmd` + `+`, `Cmd` + `-`, `Cmd` + `0`	zoom in, zoom out, fit width
`Enter`, `Shift+Enter`	send, new line
`Esc`	close a menu, cancel a note, stop marking a region

References

[1] Adam Barth, Collin Jackson, and John C. Mitchell. Robust defenses for cross-site request forgery. In *Proceedings of the 15th ACM Conference on Computer and Communications Security (CCS '08)*, pages 75–88, 2008.

[2] Patrick Billingsley. *Convergence of Probability Measures*. Wiley, second edition, 1999.

[3] Scott Chacon and Ben Straub. *Pro Git*. Apress, second edition, 2014.

[4] C. A. Ellis and S. J. Gibbs. Concurrency control in groupware systems. In *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data*, pages 399–407, 1989.

[5] A. K. Erlang. Solution of some problems in the theory of probabilities of significance in automatic telephone exchanges. *Elektroteknikeren*, 13:5–13, 1917.

[6] William Feller. *An Introduction to Probability Theory and Its Applications, Volume 1*. Wiley, third edition, 1968.

[7] James W. Hunt and M. Douglas McIlroy. An algorithm for differential file comparison. Computing Science Technical Report 41, Bell Laboratories, 1976.

[8] *ISO 32000-1:2008, Document management — Portable document format — Part 1: PDF 1.7*. International Organization for Standardization, 2008.

[9] Leonard Kleinrock. *Queueing Systems, Volume 1: Theory*. Wiley, 1975.

[10] Donald E. Knuth. *The TeXbook*. Addison-Wesley, 1984.

[11] Donald E. Knuth and Michael F. Plass. Breaking paragraphs into lines. *Software: Practice and Experience*, 11(11):1119–1184, 1981.

[12] Leslie Lamport. *LaTeX: A Document Preparation System*. Addison-Wesley, second edition, 1994.

[13] Jérôme Laurens. Direct and reverse synchronization with SyncTeX. *TUGboat*, 29(3):365–371, 2008.

[14] John D. C. Little. A proof for the queuing formula: $L = \lambda W$. *Operations Research*, 9(3): 383–387, 1961.

[15] Mozilla. PDF.js: A general-purpose, web standards-based platform for parsing and rendering PDFs. https://mozilla.github.io/pdf.js/.

[16] Eugene W. Myers. An $O(ND)$ difference algorithm and its variations. *Algorithmica*, 1(1–4): 251–266, 1986.

[17] Claude E. Shannon. Prediction and entropy of printed English. *Bell System Technical Journal*, 30(1):50–64, 1951.

[18] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, volume 6976 of *Lecture Notes in Computer Science*, pages 386–400. Springer, 2011.

[19] Frederik Tilmann. latexdiff: Determine and mark up significant differences between LaTeX files. https://ctan.org/pkg/latexdiff.